

Questor

An AI interviewer that runs first-round screens, transcribes them, and scores the evidence against the role. You decide. It doesn't.

This document has two halves. **Part One** is for anyone running screens — how to use it and what to distrust. **Part Two** is the engineering and readiness assessment: what is actually built, what is missing, and what it would take to sell this to anyone outside the company. Part Two is internal.

PART ONE

Using Questor

For recruiters, hiring managers and reviewers.

READ THIS BEFORE YOU USE IT ON A REAL CANDIDATE

This is a **pilot**. The scoring has not yet been shown to agree with human judgement — that is what this pilot exists to measure. Treat every score as a **suggestion from a colleague who sat in on the call**, not as a result. Do not reject or advance anyone because of the number.

Getting in

ADDRESS	https://questor.187-127-166-193.sslip.io (https://questor.187-127-166-193.sslip.io)
YOUR LOGIN	Ask Vishnu – you get your own account
BROWSER	Chrome or Edge

Everyone gets a personal login. Don't share one: Questor records who opened each candidate and who made each decision, and a shared account makes that record useless — which matters if a candidate ever asks who saw their interview.

There are four kinds of account. Ask for the one that matches what you actually do.

ACCOUNT	CAN DO	DELIBERATELY CANNOT
Recruiter	Create roles and candidates, send invitations, run interviews	Approve their own scorecard, or sign off an assessment
Manager	Approve scorecards, review and decide on candidates	Drive the interview itself
Reviewer	Read transcripts and record decisions	Create roles or invite candidates
Auditor	See that things happened, and when	Read candidate detail

A recruiter approving the scorecard they wrote, or signing off the interview they ran, is the check this is designed to prevent. It isn't bureaucracy — it's what keeps a human genuinely in the loop rather than nominally.

Running a screen

1

Create the role

Paste the job description. Questor reads it and proposes the competencies it will interview against, with a required level for each.

Read what it proposes. A vague JD produces vague competencies, and every question in the interview comes from this list. Five minutes here is worth more than anything else you can do.

2

Approve the scorecard

Nothing can be interviewed until a human approves the competencies. Edit the levels if they're wrong for your role — "required 4/5" on something you'd actually accept a 3 on will skew every screen after it.

3

Add the candidate

Name, email, and paste or upload their CV. You'll get a fit score against the role with the CV lines that justified it. That's a reading aid for you, not a filter — nobody is excluded by it.

4

Create the interview and send the invitation

Pick a duration; 45 minutes is the default. Questor builds a time-boxed plan across the competencies, generates a private link for that candidate, and emails it to them.

The interview screen then tells you whether it was sent and whether they've opened it. There's a **Resend email** button for when they say it never arrived.

5

The candidate takes it

They open the link, see what's being recorded and consent, do a mic check, then talk to Schranders — the AI interviewer. It asks follow-ups based on what they actually say. They can ask it to repeat, or type instead of speaking.

The link works once and expires after 14 days.

6

Review it — properly

This is the part that matters most, and it works differently to what you'd expect. See below.

Practical questions and work samples

Questor no longer only asks people to narrate their experience. On technical competencies it now puts a small **artefact** in front of the candidate and asks them to react to it — read this Flow and say what breaks at volume, look at this query, explain what this configuration will do.

Who gets one is decided by the kind of competency, not by seniority or by the role title:

COMPETENCY TYPE	WHAT THEY GET
Technical	An artefact from that competency's own domain — a Flow, a query, a config, a pipeline
Domain or situational	A realistic scenario to reason through
Communication or behavioural	Never an artefact. There is nothing meaningful to hand someone

A maximum of two per interview. The artefact always comes from the candidate's own world: a Salesforce administrator gets a Flow or a sharing model, never a Python function. That distinction is deliberate and enforced in the product, not left to the AI's discretion.

ON ASKING PEOPLE TO WRITE CODE

Writing actual code is **off by default** and has to be switched on per interview. Nobody gets a coding exercise by accident.

When it is on, be aware there is **no code editor** — the candidate types into the same answer box they speak into. That is fine for five lines and poor for anything real. Treat it as a way to see how someone thinks, not as a substitute for a proper technical round.

Reviewing: you judge first

When you open a finished interview you will **not** see the AI's score. You'll get the transcript, the exact things the candidate said against each competency, and the rating scale. You score it yourself and say why.

Only then does it show you what the AI thought, side by side, and highlight where you disagreed.

WHY IT'S BUILT THIS WAY

If you read "Proceed, 76/100" first, you can't un-read it — your judgement bends toward it without you noticing. That costs the independent read that made a second opinion worth having in the first place.

It also matters legally. An AI screening tool is treated as advisory only while a human is genuinely deciding. If reviewers habitually rubber-stamp the machine, regulators treat it as an automated decision — and the protection you were relying on is gone.

You can skip the blind step if you have a real reason — re-reading a candidate you already decided on, checking something. It asks for the reason and records it. Skipping occasionally is fine; skipping every time tells us the flow is wrong, and we'd rather know.

Your decision is the decision. Disagreeing with the AI is a useful outcome, not a problem to resolve. Every one of those comparisons is data on whether this thing can be trusted yet.

What to expect — and what not to

"Not enough evidence" is a real answer

Competencies come back unscored when the transcript doesn't support a judgement. That's the system refusing to guess, which is what you want. Those are exactly the areas to probe in round two — it's telling you where to spend your time.

A LIMITATION TO KNOW ABOUT

Evidence is currently matched to whichever competency the *question* was about. If a candidate answers question three with a great story that actually belongs to question five, that credit can be missed. So a strong candidate can show lower coverage than they deserve.

In practice: **if coverage looks low, read the transcript before concluding anything.** We tried a content-based fix and measured it as *worse* — it marked a deliberately weak transcript as a clear pass. It is switched off until it beats the current behaviour on a test we trust.

If a candidate wants to stop, it stops

Saying they're done, want to stop, or want to leave now ends the interview. The candidate gets a clean close, and **no assessment is produced** — there is no partial score sitting on their record. If someone walks out, you'll see that they withdrew, not a low mark.

You can go back to a candidate any time

Candidates and their assessments stay open from the candidates list. You can reread a transcript, revisit a decision, or send a fresh invitation to someone who never took theirs or whose link expired.

The voice is turn-based

The candidate speaks, it finishes listening, then replies. It won't interrupt or overlap the way a person does. Candidates should know it's an AI and take their time — that's in the invitation.

Things it does not do

- It doesn't decide. There is no auto-reject anywhere in the product.
- It doesn't watch or judge faces, accent, or appearance — audio and words only.
- It doesn't use anything scraped from the web. Only what the candidate gives us and what they say.
- It doesn't check fairness across groups yet. That monitoring isn't built, so we can't yet prove it scores different groups evenly.

Invitations and how to tell if they arrived

Invitation emails now send for real, from `finalerts.erats@gmail.com` . The interview screen shows one of three states:

WHAT IT SAYS	WHAT IT MEANS	WHAT TO DO
Not sent	The link exists but no email went out.	Press Resend email , or copy the link and send it yourself.
Email sent	Gmail accepted it. That is not proof it reached them.	Give it a day. Then resend, or ask them to check spam.
Opened by candidate	They loaded the link. It definitely arrived.	Nothing — wait for them to finish.

"Email sent" is not delivery. It means the mail provider took the message. Spam filters, full mailboxes and typos all happen after that point, and we don't get told. "Opened by candidate" is the only state that proves it landed.

We deliberately don't use tracking pixels, so there's no way to know they *read* it — only that they opened the link.

EXPECT SOME TO LAND IN SPAM

Invitations currently come from a `gmail.com` address and link to `questor.187-127-166-193.sslip.io` — a temporary address while we pilot. The connection is properly encrypted and browsers trust it, but that combination is exactly what spam filters and cautious candidates distrust.

Tell candidates to expect the email, ideally on the call or message where you agree to interview them. A company address is coming.

FIXED 19 JULY — LINKS USED TO FAIL ON SOME NETWORKS

Until this evening Questor answered on port `8443`, a non-standard port that some corporate networks, guest wi-fi and mobile carriers drop. A candidate on one of those networks got *"this site can't be reached"* on a link that worked perfectly for everyone else — and because the connection never arrived, nothing appeared in our logs to explain it. One real candidate hit this before we found it.

It now answers on the standard HTTPS port, and older `:8443` links redirect there automatically. **If a candidate still reports not being able to connect, tell us** — that would be a different fault, and we'd want to see it immediately.

What we need from you

Use it on real screens where you'd have done a first round anyway, then tell us:

- **Did you agree with it?** Especially where you didn't — that's the most valuable thing you can report.
- **Were the questions any good?** Would you have asked them?
- **Did anything feel unfair** to a candidate, or would you be uncomfortable if they saw it?
- **What did candidates say** about the experience?
- Anything confusing, broken, or slow.

Agreement between your calls and the AI's is being tracked automatically. Every review you complete is one data point toward knowing whether this can be trusted. Until there are enough of them the honest position is: we don't know yet

whether it scores well, and nobody should act as if we do. *How* many is enough is covered in Part Two, and the number is larger than we first thought.

Engineering and enterprise readiness

Not for circulation to candidates, customers or prospects.

WHY THIS HALF EXISTS

Part One describes a product that works. This part describes what is holding it up, and it is thinner than the first half implies. Everything below was found by reading the running code and the live server, not by inference.

Where it actually stands

CLAIM	STATUS
Interview runs end to end by text	Verified live on the deployed instance
Withdrawal produces no score	Verified live
Blind-first review enforced	Enforced at the route, not just the UI
Test suite	242 passing, five consecutive clean runs from committed state
Voice path, end to end, by a human	Never completed. Every verification so far drove the API with text
Scoring agrees with human judgement	Unproven. $n \approx 1$
Fairness across groups	Not computed. The monitoring is a stub
Candidates invited	Five, live now

The gap between rows one to four and rows five to seven is the whole risk. The machinery is sound; nothing yet says the output is right.

Architecture gaps

None of these are sloppiness. They are the seams of a well-built single-tenant appliance that was never cut into a product. Ordered by whether they would block a sale.

GAP	WHAT BREAKS
No migrations directory — <code>db:push</code> is the deploy model	A customer's database cannot be upgraded deterministically or rolled back, and <code>db:push</code> silently drops columns. One bad release is unrecoverable data loss. Everything else on this list needs this fixed first
No ATS integration, and no <code>externalId</code> field on Candidate or Role	Deal-ending. No HR organisation buys a screening tool that cannot write back to their system of record. The Greenhouse connector is an empty subclass; there is nowhere to store a Workday worker id
Assessments are mutable; no unique constraint on <code>(sessionId, version)</code>	In a discrimination claim, "can you prove this assessment was not altered after the decision?" has no answer. Two concurrent finalises can both claim version 1
No timeout on the LLM call	Node's default is 300 seconds. A hung provider leaves a candidate on a blank screen for five minutes. The pattern exists elsewhere in the codebase — it was just never applied to the candidate-facing path
Silent fallback to template questions	When the model is unavailable the interview quietly continues on a canned question bank and <i>nothing records that it did</i> . Two candidates for the same role, one interviewed by the model and one during an outage, produce assessments that look identical. That is an adverse-impact problem, not a quality one
SQLite in rollback-journal mode, no <code>busy_timeout</code>	Every write locks the whole file. Under contention the error surfaces as a generic failure and the candidate's answer is discarded — silently, with no recruiter-visible trace
Turn index race, no unique constraint	Both the socket and HTTP paths write turns with no mutual exclusion. A double-click can produce duplicate indices and a permanently scrambled transcript — which is the evidence record behind a hiring decision
No index on <code>Turn.sessionId</code>	Every candidate answer full-scans the turn table, twice. Degrades with history, not just concurrency
The 999-variable <code>IN</code> - clause ceiling	List endpoints materialise every id in the tenant. A customer with more than ~999 sessions gets 500s on every page — broken, not slow. The correct pattern already exists elsewhere in the same file
Multi-tenancy enforced only in application code	No row-level security, no middleware. Consistently applied today, but every query is one forgotten clause from a cross-tenant leak, and the only guard is developer memory

GAP	WHAT BREAKS
<code>User.email</code> is globally unique	One person cannot exist in two tenants — a functional blocker for staffing agencies and consultancies, and a schema change that gets harder with every customer
<code>ModelExecution</code> has no <code>tenantId</code>	AI cost cannot be attributed to a customer. No usage billing, no per-customer spend reporting, no answer to "who is burning the budget"
In-memory state: no <code>Socket.IO</code> adapter, rate limiter is a module-level map	The app is structurally single-process and <i>nothing fails loudly</i> if it is scaled out. Two workers means observers see nothing and every spend limit doubles

Operations gaps

THE BACKUP HAS NEVER RUN

The cron entry exists and the script itself is well written — it uses SQLite's own backup API, verifies integrity before trusting the file, and prunes on a retention window. But `backup.log` does not exist on the server, and shell redirection creates that file the instant the job fires *even when it fails*. Its absence is proof the job has never executed.

The only backup that exists is a single manual one, taken before any of the current candidates were invited. **Every interview recorded since then exists in exactly one place.** There is also no offsite copy — the backup directory is on the same disk as the live database — and no restore has ever been tested.

GAP	CONSEQUENCE
No error tracking or alerting of any kind	If a candidate's interview fails at 2am, nobody finds out. The only detection path is a person opening the logs by hand
No <code>uncaughtException</code> handler	A synchronous crash kills the process with a stack trace in a local file and no notification
Health endpoint returns a hardcoded "ok"	It does not check the database, so it reports healthy while the app is unable to serve a single interview. Nothing external polls it anyway
Deploy is stop-then-start with no <code>SIGTERM</code> handler	Deploying kills every live interview outright — no drain, no warning, no resume. There is no repeatable deploy script and no rollback mechanism
No aggregate spend cap	Per-candidate rate limits are good, but nothing bounds total AI spend and nothing alerts on it. A bulk hiring push has no ceiling
Deploy knowledge lives in one person's head	Several operational scripts exist only on that one disk, untracked. Nobody else can reproduce the deployment

Single points of failure

- One machine holds the app, the live database, *and* the only backup.
- One process, no redundancy, no graceful drain — any crash or deploy drops every active interview.
- SQLite is single-writer with no replica.
- One API key drives the model, speech-to-text and speech — one outage stops everything. Alternative providers exist in the code but are not wired as automatic failover.
- Routing depends on a third-party IP-to-hostname convenience service, on a non-standard port, not an owned domain.
- Detection of any failure depends entirely on a human deciding to look.

What blocks a sale

A vendor-security review at any large company would stop at the first page. In rough order of how early it comes up:

- **No SOC 2.** The Type II report needs an observation window of six to twelve months. It cannot be compressed by spending more.
- **No independent penetration test**, and no remediation letter to show afterwards.
- **No Data Processing Agreement**, no subprocessor list, no data-flow diagram, no incident-response policy.
- **No SSO.** No SAML or OIDC, no SCIM provisioning, no enforced MFA. Enterprises do not accept per-user passwords for a system holding candidate data.
- **No data residency story.** EU buyers will ask where candidate data physically sits before they ask anything else.
- **No trust centre, no references, no SLA, no cyber insurance.**

Regulatory position

Some of this is materially better than we assumed, and one part is worse. The dates below reflect changes through 2026 and are worth re-checking before anything is committed contractually.

REGIME	WHERE IT STANDS
EU AI Act	Employment screening is high-risk regardless of the tool being advisory. But the deadline for these obligations was deferred to 2 December 2027 , and employment systems qualify for internal conformity assessment — self-declared, with no notified body required. Far cheaper and slower-moving than feared. Still needs a quality management system, risk management, technical documentation, logging, human oversight and post-market monitoring
NYC Local Law 144	Live now, and the one that bites. Requires an annual bias audit by an independent assessor, published publicly, plus ten business days' notice to candidates. Penalties run \$500–\$1,500 per day, per tool . Enforcement scrutiny increased after a 2025 audit found city oversight ineffective. Applies the moment a New York candidate is screened
Colorado	SB 24-205 was repealed and replaced by SB 26-189, effective 1 January 2027 and scaled back — disclosure, transparency, and a right to human review, rather than a full risk-management programme
Illinois	HB 3773 took effect 1 January 2026 : bans ZIP code as a proxy for race and mandates candidate notice. AIVIA additionally applies if video is ever used — it is not today
India (DPDP)	Consent, purpose limitation and security obligations on candidate data. Large customers will push impact-assessment and audit duties down to us contractually

TWO CASES WORTH WATCHING

Mobley v. Workday is in discovery on the theory that the *vendor* — not only the employer — is liable for discriminatory screening. That is the case that makes this a product risk rather than a customer risk.

Kistler v. Eightfold (filed January 2026) opens a different front entirely: that a screening vendor scraping web and social data is acting as an unregistered consumer reporting agency under the FCRA. Questor is well positioned here by construction — it uses *no scraped data at all*. That is worth keeping true deliberately.

Validity: the number was wrong

We have been working to "about 30 reviewed interviews". That figure is too low for what it was being used to claim.

A defensible **criterion validity** study — showing scores actually predict job performance — needs **N = 100 to 150** under the federal selection guidelines. Below 50 the result is not statistically meaningful, and a study that small is worse than none because it invites a claim it cannot support.

The realistic route at our volume is **content validity** instead: a formal job analysis for each role family, mapping every rubric competency to documented task and knowledge requirements. That is achievable now, does not depend on candidate volume, and is a recognised basis for defending a selection procedure.

Thirty paired blind reviews remains worth collecting — it gives inter-rater reliability and tells us whether reviewers agree with the machine and with each other. It just is not a validation study, and should never be described as one. Separately, **adverse impact** monitoring under the four-fifths rule needs building; it is currently a stub, and computing it honestly requires demographic data we do not lawfully hold today.

Competitive position

Trying to beat HireVue, Sapia or Karat at being an enterprise AI interview platform would fail. They have SOC 2 Type II, years of validation data, published external bias audits, industrial psychology positioning, and procurement collateral. On enterprise trust and validation corpus this is not a contest.

The defensible position is narrower and genuinely available:

- **Zero scraped data.** Only what the candidate provides and says. Directly relevant given the FCRA line of attack now open against scraping vendors.
- **Explainable rubrics with quoted evidence**, rather than an opaque match percentage — which is also what the emerging right-to-explanation rules ask for.
- **Human decides, structurally.** Blind-first review is enforced in the routes, not a setting. That is a real differentiator, and it is already built.
- **Self-auditing.** A tool that computes adverse impact for the customer on their own applicant pool, and warns them before a regulator does, is something smaller employers will pay for and incumbents sell as a consulting add-on.

The honest one-line positioning today: *a controlled pilot system for structured candidate evidence collection with advisory review*. The sellable version: *compliance-conscious, human-first work-sample screening for employers who want structure and auditability without handing hiring judgement to a black box*. What must never be claimed is that it ranks candidates fairly — nothing supports that yet.

The dissenting view on fixing this quickly

An initial plan proposed fixing the most dangerous items immediately — switch the database to write-ahead logging, add a timeout, add the missing unique constraint, reschedule the retention sweep. Adversarial review took that plan apart, and the objections were right:

PROPOSED FIX	WHY IT IS NOT THAT SIMPLE
Set write-ahead logging and a busy timeout at startup	Both settings are per connection , not global. Applying them once at boot may not survive the pooled connections the query engine opens later, so the fix can appear applied and do nothing. Write-ahead mode also creates sidecar files that a naive file-level backup will miss — meaning recent interview turns could silently vanish from backups
Add the unique constraint on turn index	The deploy model is <code>db:push</code> with no migration history. On SQLite, adding a constraint can mean recreating the table underneath — which can lock or fail midway on a live database. It also fails outright if duplicates already exist, so it needs a preflight duplicate check. Schema surgery cannot be deferred-and-also-performed : either migrations get baselined first, or this waits
Twenty-second timeout plus one retry	That is a forty-second wait, not twenty. Worse, a request that times out locally may still have completed upstream, so the retry can produce a different next question and leave the transcript holding an answer to a question nobody sees
Give the retention sweep a window and jitter	Jitter solves a distributed problem we do not have. On one machine the sweep either collides with a live interview or it does not. The correct immediate action is to disable it , not reschedule it

THE LARGEST IMMEDIATE RISK IS DEPLOYING THE FIX

With no drain handler, **a deploy terminates every interview in progress**. While candidates hold live links, shipping five changes at once — a storage-mode switch, a schema constraint, timeout logic, new metadata and a scheduler change — risks doing more damage than the bugs would, and leaves no way to tell which change caused what. Rolling back is not clean once the database schema has been altered.

The safer immediate set is smaller and touches no schema: disable the retention sweep, add a per-session lock in the application to serialise turn writes, put a total deadline on the model call with a visible "still working" state, and confirm the backup actually runs.

Sequenced roadmap

A • Safe to do now, no schema changes

- Confirm the backup cron fires, then take one immediately by hand and copy it **off the machine**.
- Set a hard monthly spend cap and alert on the AI provider's dashboard. Five minutes, no code, no deploy.
- Disable the retention sweep in production.
- Add error tracking and an external uptime check against a health endpoint that actually queries the database.
- Add an `uncaughtException` handler so a crash restarts cleanly instead of hanging broken.
- Move to the standard HTTPS port so blocked corporate and mobile networks can reach it.

B • Before the next batch of candidates

- A drain handler, so a deploy stops accepting new sessions and lets live interviews finish.
- A per-session lock serialising turn writes across both transports.
- A total deadline on the model call, with a visible waiting state, plus a circuit breaker.

- Record when an interview ran on fallback questions, and surface that in the assessment's limitations.
- Baseline migrations. Nothing structural can be fixed safely until this exists.
- Test a restore. A backup nobody has restored is a hypothesis.
- Have someone complete the voice path end to end — still the outstanding condition.

C • Before selling to a first customer

- Assessment immutability: content hashes, unique version constraints, an append-only audit trail.
- One real ATS connector built properly, plus `externalId` on candidates and roles. One deep integration beats three shallow ones.
- Move to a managed database with encrypted volumes and point-in-time recovery, off the shared machine. The schema was deliberately written to be portable, so this is mostly operations work.
- SSO, role-based access, exportable per-candidate audit trail.
- An owned domain and a transactional email provider with proper sender authentication.
- The procurement pack: DPA, subprocessor list, data-flow diagram, retention policy, incident response, lawyer-reviewed candidate notices.
- An independent penetration test, with the findings fixed and a retest letter.
- Content-validity job analyses for the role families in use.

D • Before an enterprise buyer

- SOC 2 Type II. The observation window is six to twelve months and cannot be shortened, so this is the item that sets the calendar — start the controls early even if the audit starts late.
- ISO 27001, and ISO 42001 for AI management if selling into Europe.
- Real adverse-impact reporting, and an independent bias audit if any New York candidate is ever screened.
- Multi-tenancy enforced structurally rather than by convention; per-tenant cost attribution.
- Horizontal scale: shared session state, distributed rate limiting, externalised scheduling.

- An EU AI Act technical file, self-declared under internal conformity assessment, ahead of December 2027.

ROUGH SHAPE OF THE EFFORT

Tier A is an evening. Tier B is days. Tier C is the better part of a quarter of focused work plus external spend on the penetration test and legal review. Tier D is six to twelve months, governed almost entirely by the audit observation window rather than by engineering.

Questor pilot · questions and access requests to Vishnu.

Part One may be shared with the hiring team. Part Two is internal — it should not go to candidates, customers or prospects.

Candidate transcripts and CVs are real personal data — don't copy them out of the tool.